Experiment No. 02

Aim: To study and prepare a detailed Software Requirement Specification (SRS) document for the DocDigest project, covering its functional and non-functional requirements for secure user authentication, PDF document management, document storage, retrieval, and future AI-assisted document processing.

Problem Statement:

Handling a large number of digital documents becomes difficult when users need a proper way to store, organise, search, and access them safely. Regular file storage systems mostly just offer storage and download options, without any real application-level login system or ownership-based access control. Because of this, there is a need for a system that lets users manage their documents securely, making sure that no one else can access their personal files. DocDigest is proposed as a web-based document management application where users can register and log in securely using JWT-based authentication, upload PDF documents, store their files along with related metadata, and access only the documents that belong to them. The system is also built to support PDF text extraction and AI-assisted features such as document summarisation and flashcard generation in later phases, so that users can go through lengthy documents more easily.

SOFTWARE REQUIREMENT SPECIFICATION (SRS)

1. Introduction

1.1 Purpose - The purpose of this SRS document is to define the functional and non-functional requirements of DocDigest, a secure web-based document management system. It acts as a common reference point for the analysis, design, development, testing, and future improvement of the project.

1.2 Scope

DocDigest gives authenticated users a platform to upload, store, retrieve, and download PDF documents. User authentication is handled through JWT, and document ownership is checked before any protected operation is carried out.

1.2.1 In Scope - User registration and login, password hashing, JWT-based authentication, PDF uploading, filesystem storage, document metadata management, user-document association, retrieval of user-specific documents, and secure document downloading.

1.2.2 Out of Scope (Current Phase) - AI-based summarisation, flashcard generation, advanced content-based search, support for file formats other than PDF, cloud-based file storage, and advanced document organisation are all planned for later phases.

1.3 Intended Audience

This document is meant for end users, especially students and professionals who need to manage digital documents; developers and testers who will build and test the system; and project guides and evaluators reviewing the project.

1.4 Benefits of the System

1.4.1 Social Benefits - DocDigest gives users an organised and secure way to manage their digital documents and cuts down the effort needed to find and access frequently used files.

1.4.2 Environmental Benefits - The system promotes digital document storage and reduces dependence on printed documents and physical file storage.

1.4.3 Economical Benefits - The system reduces the time and effort spent on manually managing documents and relies mostly on open-source technologies, which keeps the development cost low.

1.5 Document Overview

This document describes the overall system, system requirements, functional and non-functional requirements, security considerations, software attributes, budget, references, and conclusion of the DocDigest project.

2. Overall Description

2.1 Product Perspective - DocDigest is an independent web-based application built using Java and Spring Boot. The client communicates with the backend through REST APIs. User information and document metadata are stored in MySQL, while the actual PDF files are kept in the configured filesystem location.

The application uses Spring Security along with JWT to authenticate users and protect document-related operations.

2.1.1 - The system follows a layered architecture made up of controller, service, repository, security, and database components.

2.1.2 - The backend takes care of user authentication, authorisation, document uploading, retrieval, downloading, and ownership verification.

2.1.3 - MySQL stores user and document metadata, while uploaded PDF files are kept separately in filesystem storage.

2.1.4 - The architecture is designed so that PDF processing and AI-based features can be added later without making major changes to the existing authentication and document management modules.

2.2 Product Functions

- Allow users to register and log in securely.
- Generate and validate JWT tokens for authenticated requests.
- Allow authenticated users to upload PDF documents and store their metadata.
- Allow users to view and retrieve only the documents that belong to their account.
- Allow users to download their stored PDF documents.
- Provide a foundation for future PDF text extraction, document search, AI summarisation, and flashcard generation.

2.3 User Characteristics

2.3.1 General Users - Users with basic computer and internet knowledge who need to upload and manage PDF documents.

2.3.2 Students - Students may use DocDigest to store notes, textbooks, question papers, assignments, and other academic material.

2.3.3 Professionals - Professionals may use the system to manage reports, manuals, technical documents, and other work-related files.

2.3.4 Researchers - Researchers may use the system to organise research papers and other academic documents.

2.3.5 Developers and Testers - Technical users responsible for developing, testing, maintaining, and extending the application.

2.4 Constraints

2.4.1 - The current system mainly supports PDF documents.

2.4.2 - Authentication is required before any protected document operation can be accessed.

2.4.3 - Users can access only the documents associated with their own accounts.

2.4.4 - The maximum file size and total storage depend on application configuration and available storage space.

2.4.5 - Future AI functionality will depend on the AI service or model chosen and the resources available for it.

2.5 User Documentation

2.5.1 - Basic instructions should be provided for registration, login, uploading, retrieving, and downloading documents.

2.5.2 - API documentation may be maintained for developers and testers so they can understand and test the available REST endpoints.

2.6 Dependencies

2.6.1 - The system depends on Java, Spring Boot, Spring Security, Spring Data JPA, Hibernate, MySQL, and JWT libraries.

2.6.2 - The application depends on available filesystem storage for the uploaded PDF files.

2.6.3 - Future PDF processing and AI features will depend on the PDF processing library and AI service that are chosen.

3. System Requirements and Analysis

3.1 User Interface Requirements

The system should provide a simple interface through which users can register, log in, upload PDFs, view their documents, retrieve document information, and download files. Clear messages should be shown for successful operations as well as for errors such as invalid credentials, failed uploads, or unavailable documents.

3.2 Hardware Requirements

3.2.1 Client-Side Requirements - A computer, laptop, tablet, or any compatible device with a modern web browser and an internet connection.

3.2.2 Server-Side Requirements - A system capable of running Java and Spring Boot, with enough RAM, processor capacity, network connectivity, and storage for the application, database, and uploaded PDF files.

3.2.3 - The exact hardware requirements may go up depending on the number of users, the number of stored documents, and future PDF/AI processing workloads.

3.3 Software Requirements

The following software and technologies are used or planned for DocDigest:

- Programming Language: Java 21
- Backend Framework: Spring Boot 4.1.0
- Web Framework: Spring WebMVC
- Database: MySQL
- Database Access: Spring Data JPA / Hibernate
- Security: Spring Security 7.1.0
- Authentication: JWT
- Password Security: BCrypt
- Object Mapping: ModelMapper
- Build Tool: Maven
- Development Environment: IntelliJ IDEA
- API Testing: Postman
- Version Control: Git / GitHub
- PDF Processing: Planned
- AI Processing: Planned

3.4 Communication Interfaces

The system uses HTTP/HTTPS-based REST APIs for communication between the client and the backend. JWT is used to authenticate requests. HTTPS should be used in production to protect credentials, tokens, and document-related data while they are being transmitted.

4. Supplementary Requirements

4.1 - The system shall maintain a clear separation between users and their documents so that one user cannot access another user's files.

4.2 - The application should provide proper error handling and logging to help identify and fix failures.

4.3 - The system should stay modular so that future features such as PDF text extraction, search, summarisation, and flashcard generation can be added with minimal changes to the existing modules.

5. Functional Requirements

5.1 User Registration and Authentication

5.1.1 User Registration - The system shall allow new users to create an account using the required registration details.

5.1.2 User Login - The system shall authenticate registered users using their credentials.

5.1.3 Password Security - User passwords shall be securely hashed using BCrypt before being stored, and shall never be stored in plain text.

5.1.4 JWT Authentication - After a successful login, the system shall generate a JWT token. This token shall be validated for every protected request that follows.

5.1.5 Authorization - Only authenticated users shall be allowed to access protected document operations.

5.2 Document Upload and Storage

5.2.1 PDF Upload - Authenticated users shall be able to upload PDF documents.

5.2.2 File Storage - Uploaded PDF files shall be stored in the configured filesystem location.

5.2.3 Metadata Storage - The system shall store relevant metadata such as document title, file name, file size, upload date, file path, and document owner.

5.2.4 User Association - Each document shall be linked to the authenticated user who uploaded it.

5.2.5 Unique File Naming - A unique identifier shall be used while storing uploaded files so that filename conflicts are reduced.

5.3 Document Retrieval and Management

5.3.1 - Users shall be able to retrieve a list of documents that belong to their account.

5.3.2 - Users shall be able to retrieve information about a specific document that they own.

5.3.3 - The system shall prevent users from accessing documents that belong to other users.

5.3.4 - Users shall be able to download their stored PDF documents.

5.3.5 - The system shall provide document deletion functionality.

5.3.6 - When a document is deleted, both its physical file and its database metadata should be removed.

5.4 Future Document Search and Organization

5.4.1 - Future versions may allow document search using metadata such as title or file name.

5.4.2 - Future versions may support content-based search using extracted PDF text.

5.4.3 - Future versions may add features such as sorting, pagination, categories, and tags.

5.5 Future PDF Text Extraction

5.5.1 - The system is intended to extract text from supported PDF documents in a future phase.

5.5.2 - The extracted text may be used for document search and AI-assisted processing.

5.6 Future AI-Assisted Processing

5.6.1 Document Summarization - The system is intended to generate concise summaries from PDF content in a future phase.

5.6.2 Flashcard Generation - The system is intended to generate study-oriented flashcards from document content in a future phase.

5.6.3 - AI-generated content shall be accessible only to the authenticated user who has access to the corresponding document.

6. Non-Functional Requirements

6.1 Performance Requirements

6.1.1 Usability - The system should provide a simple interface that lets common document operations be carried out without needing much technical knowledge.

6.1.2 Security - Authentication, authorisation, password hashing, and protected API access shall be used to stop unauthorised access.

6.1.3 Reliability - The system should keep document metadata and the corresponding stored files consistent during normal operation.

6.1.4 Accessibility - The interface should use readable text, clear labels, simple navigation, and error messages that are easy to understand.

6.1.5 Portability - The backend should run on operating systems that support the required Java runtime and application dependencies.

6.1.6 Software System Attributes - The application should follow a modular, layered architecture to make development, testing, maintenance, and future enhancement easier.

6.1.7 Safety - User credentials, authentication information, document metadata, and stored documents should be protected from unauthorised access.

6.2 Security Requirements

6.2.1 - Spring Security shall be used to implement authentication and authorisation.

6.2.2 - JWT shall be used to authenticate requests made to protected REST API endpoints.

6.2.3 - Passwords shall be hashed using BCrypt before they are stored in the database.

6.2.4 - The system shall verify document ownership before allowing retrieval, downloading, or deletion of a document.

6.2.5 - Document ownership shall be determined from the authenticated user, and not from an untrusted user ID sent by the client.

6.2.6 - HTTPS should be used in production to protect sensitive information during transmission.

6.3 Portability Requirements

6.3.1 - The backend application shall be able to run on operating systems that support Java 21 and the required Spring Boot environment.

6.3.2 - The application should be deployable on suitable local or cloud-based server environments.

6.4 Accessibility Requirements

6.4.1 - The interface should provide clear navigation and controls that are easy to understand for common document operations.

6.4.2 - Error messages and system feedback should be easy to understand.

6.5 Reliability Requirements

6.5.1 - The system should keep the physical PDF files and their corresponding database records consistent.

6.5.2 - Invalid requests and unexpected errors should be handled without causing the whole application to fail.

6.5.3 - Proper exception handling and logging should be used to help identify application failures.

6.5.4 - Future production deployment should include regular backups of the database and document storage.

6.6 Usability Requirements

6.6.1 - The registration and login process should be simple and easy to understand.

6.6.2 - Document uploading should require only the necessary information and should give clear feedback once it is done.

6.6.3 - The system should clearly show whether document operations such as retrieval, download, or deletion were successful.

6.7 Software System Attributes

6.7.1 Maintainability - The codebase shall follow a layered and modular structure to make maintenance and debugging simpler.

6.7.2 Scalability - The architecture should allow the number of users and documents to grow without needing a complete redesign.

6.7.3 Modularity - Authentication, document management, storage, and future processing modules should stay logically separated.

6.7.4 Extensibility - The system should allow future integration of PDF text extraction, advanced search, AI summarisation, flashcard generation, and other document-processing features.

6.8 Safety Requirements

6.8.1 - Uploaded files and incoming requests should be validated to reduce invalid or potentially harmful input.

6.8.2 - Document ownership shall be verified before any protected document operation is carried out.

6.8.3 - Failures during document storage or processing should be handled without leaving behind inconsistent database records or inaccessible files.

7. Preliminary Budget

The table below gives a rough estimate of the resources needed for developing and running DocDigest. Since the project mostly uses open-source technologies, the development cost is expected to stay low. The actual cost may change if paid hosting, storage, or AI services are used later.

S.No	Item	Description	Estimated Cost
1	Development Tools	IntelliJ IDEA (Community Edition), Postman, Git/GitHub	Free (Open Source)
2	Backend Technologies	Java, Spring Boot, Spring Security, Hibernate	Free (Open Source)
3	Database	MySQL	Free (Open Source)
4	File Storage	Local filesystem storage	Free (existing hardware)
5	Hosting (for demo/deployment)	Cloud or local server used for demonstration	Rs. 0 - 1000/month (optional)
6	Domain Name	Required only if the project is deployed publicly	Rs. 800/year (optional)
7	Internet & Miscellaneous	Internet usage, printing, report preparation	Rs. 500 (approx.)
Total Estimated Cost			Rs. 500 - 2300 (approx.)

8. Appendices

8.1 References

8.1.1 - IEEE Recommended Practice for Software Requirements Specifications (IEEE 830), referred to for the structure and preparation of this SRS.

8.1.2 - Official documentation for Java and Spring Boot, used as a reference for application development.

8.1.3 - Official documentation for Spring Security and JWT, used for implementing authentication and authorisation.

8.1.4 - Official documentation for MySQL, Spring Data JPA, and Hibernate, used for database management and persistence.

8.1.5 - Project-specific requirements and design decisions made during the development of DocDigest.

9. Conclusion

This Software Requirement Specification lays out the purpose, scope, users, system requirements, functional requirements, and non-functional requirements of DocDigest. The current system is focused on secure user authentication and PDF document management, which includes registration, JWT-based login, document uploading, filesystem storage, metadata management, user-specific retrieval, and document downloading.

The project uses Java, Spring Boot, Spring Security, JWT, MySQL, and Spring Data JPA/Hibernate in a layered, modular architecture. The system is designed so that it can be extended in later phases with PDF text extraction, advanced search, AI-based summarisation, and flashcard generation.

This SRS gives a structured base for the design, development, testing, and future improvement of DocDigest, while also covering important aspects such as security, reliability, maintainability, usability, and scalability.